



C프로그래밍 (CSE2003)

4-2. Functions

함수 호출(function call)

- Parameter passing
 - ♦ C 언어에서는 함수를 호출할 때 argument 전달하는 방법 두 가지가 있다.
 - 값에 의한 전달 (**Pass by Value**)
 - 직접 값을 전달하는 방법
 - 호출한 함수의 값이 변하지 않는다.
 - 참조에 의한 전달 (**Pass by Reference**)
 - 주소값을 전달하는 방법
 - 호출한 함수의 값이 변한다.

예제 프로그램 2 – Pass by value

예제 소스

```
#include <stdio.h>

void add(int first, int
        second);
int
main(void)
{
    int first;
    int second;

    first = 5;
    second = 10;

    add(first, second);
    printf("first = %d, second = %d\n", first, second );

    return 0;
}

void add(int first, int second)
{
    first = first + second;
}
```

add() 함수 내에서 계산이 이루어
지지만 그것은 복사본의 계산일 뿐이다.

호출하는 쪽의 인자와 호출되는
쪽의 인자는 서로 다른 메모리에
저장되어 있어 아무 관계가 없다.

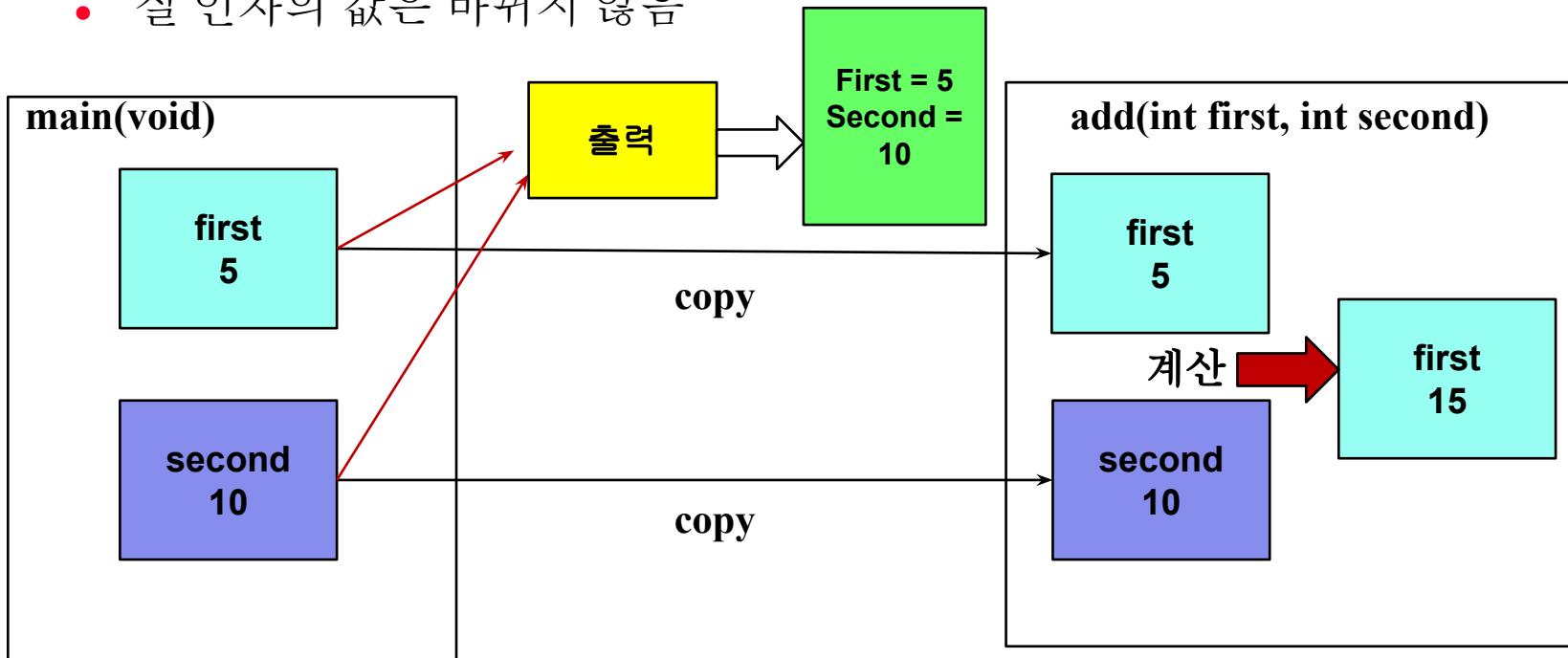
결과

```
kimjihwan@csp:~/test_dr$ gcc -o pass.o pass.c
kimjihwan@csp:~/test_dr$ ./pass.o
first = 5, second = 10
kimjihwan@csp:~/test_dr$
```

Pass by value

■ Pass by value (Call by value)

- 값에 의한 전달이란 뜻은 Caller에서 함수를 호출할 때 전달하는 argument로 값(value)을 사용한다는 것
- 형식 인자와 실 인자가 **따로 메모리를** 가짐
- 실 인자의 값을 형식 인자에 복사하여 사용
- 실 인자의 값은 바뀌지 않음



예제 프로그램 1 - Pass by reference

```
#include <stdio.h>

void add(int * first_addr, int*
        second_addr);

int
main(void)
{
    int first;
    int second;

    first = 5;
    second = 10;

    add(&first, &second);
    printf("first = %d, second = %d\n", first, second );

    return 0;
}

void add(int* first_addr, int* second_addr)
{
    *first_addr = *first_addr + *second_addr;
}
```

main은 add()를 호출하면서 변수 first와 second의 주소 넘겨줌
add()는 이를 포인터(주소를 저장하는 변수) first_addr, second_addr에 받음
두 값을 더함
실행결과 main함수에서 두 변수의 값이 바뀌는 것을 확인할 수 있음

결과

```
kimjihwan@cspro:~/test_dr$ gcc -o pass.o pass.c
kimjihwan@cspro:~/test_dr$ ./pass.o
first = 15, second = 10
kimjihwan@cspro:~/test_dr$
```

Pass by Reference

- Pass by Reference (Call by Reference)
 - 참조에 의한 전달은 호출되는 함수에 argument의 값을 전달하는 것이 아니라, argument의 **메모리 주소(address)값을 전달**하는 것
 - 메모리 주소 값을 전달하면 호출되는 함수는 그 주소 값을 이용하여 **실 인자의 값을 바꿀 수 있음**
- Address operator (&) : “Give me the address of this data”
- 변수의 주소를 얻어냄
- caller 측에서 사용 (주소를 구해서 callee에게 넘겨줌)
- Indirection operator (*) : “Use this value as an address to find the data”
- 주소에 해당하는 위치(또는 그곳의 값)를 나타냄
- callee 측에서 사용 (넘겨받은 주소를 이용해서 변수에 access)
- Indirection operator와 address operator는 반대 개념

예제 프로그램 2 - 나눗셈

```
#include <stdio.h>

void Calc_divide(int in1, int in2, int*
result);
int Calc_remainder(int in1, int in2);
int
main(void)
{
    int intNum1;
    int intNum2;
    int intCalc;

    printf("Enter two integral number: ");
    scanf("%d %d", &intNum1, &intNum2);

    Calc_divide(intNum1, intNum2, &intCalc);
    printf("%d / %d is %d", intNum1, intNum2, intCalc);

    intCalc = Calc_remainder(intNum1, intNum2);
    printf(" with a remainder of: %d\n", intCalc);

    return 0;
}

void Calc_divide(int in1, int in2, int* result)
{
    *result = in1 / in2;
}

int Calc_remainder(int in1, int in2)
{
    int result_remainder;
    result_remainder = in1 % in2;
    return result_remainder;
}
```

예제 소스

```
kimjihwan@cspiro:~/test_dr$ gcc -o calc.o calc.c
kimjihwan@cspiro:~/test_dr$ ./calc.o
Enter two integral number: 13 2
13 / 2 is 6 with a remainder of: 1
kimjihwan@cspiro:~/test_dr$
```

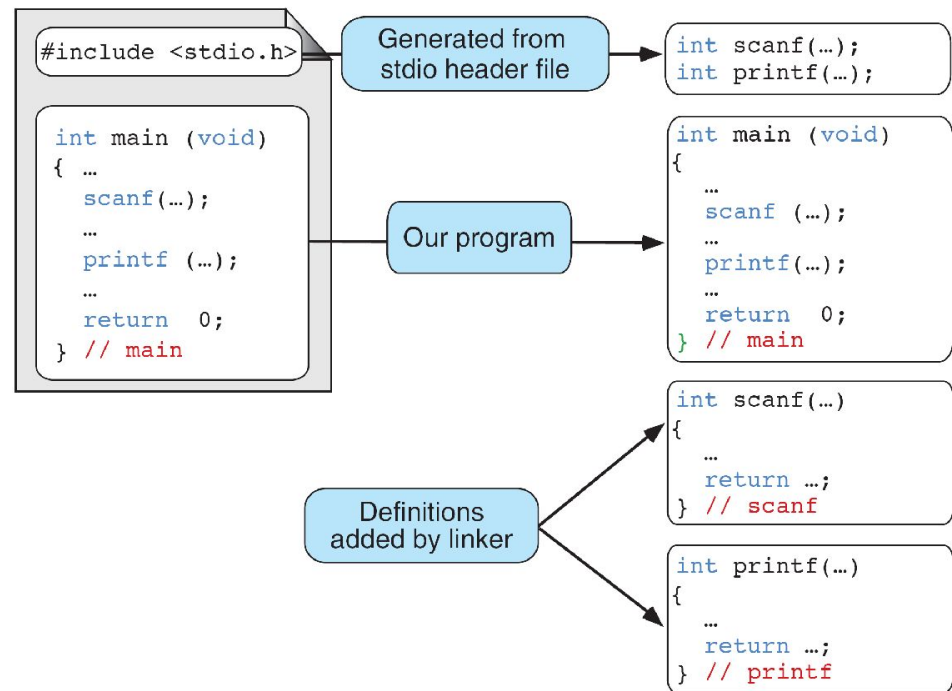
결과

Standard Functions

- 일반적으로 널리 사용하는 기능들을 함수로 정의하여 제공하는 라이브러리
- Function declaration 을 가지고 있음
- 다양한 라이브러리를 제공함
- 사용자도 라이브러리를 만들어 사용할 수 있음
- `<stdio.h>`
 - `printf()`, `scanf()`
- `<math.h>`
 - `double fabs (double number)`, `double ceil (double number)`, `double floor (double number)`, `double pow (double x, double y)`, `double sqrt (double number)`
- `<stdlib.h>`
 - `int abs (int number)`, `long labs (long number)`, `void srand(unsigned int seed)` // seed 제공, `int rand (void)` // 0에서 `RAND_MAX` 값 사이의 정수가 나옴

Standard Functions

- C언어에는 프로그래머의 편의성을 도모하기 위해
 - 프로그램 언어 개발자들에 의해 작성되어 포함된 함수들이 존재
 - 널리 사용하는 기능들을 함수로 정의하여 제공하는 라이브러리
 - Standard Functions를 불러 들이기 위해 전처리 구문 영역에 `#include`문을 사용
 - ◆ 대표적인 표준 라이브러리 함수인 `scanf()`와 `printf()` 함수를 사용하기 위해 `<stdio.h>` 헤더파일 이용
 - 대표적인 종류
 - ◆ 표준 입출력 함수 `<stdio.h>`
 - ◆ 문자열 조작 함수 `<string.h>`
 - ◆ 문자 관련 함수 `<ctype.h>`
 - ◆ 유틸리티 함수 `<stdlib.h>`
 - ◆ 시간 및 날짜 관련 함수 `<time.h>`
 - ◆ 수학 관련 함수 `<math.h>`



실습 9

실습 10

Scope

- 프로그램 내에서 정의한 변수 (또는 상수, 함수이름, function declaration)가 유효한 범위를 scope 라 함
 - 예를 들면, main 함수 안에서 선언한 변수의 scope(그 변수를 사용할 수 있는 범위)은 main 함수 내부
- 유효 범위에 따라 크게 두 가지로 구분
 - 지역(local) 변수 : main함수포함 모든 함수들의 안에서 정의
 - 전역(global) 변수 : main함수포함 모든 함수들의 밖에서 정의
- 변수들은 그것이 정의된 함수나 블록 안에서만 유효
 - 즉, 함수 안에서 선언된 변수(지역변수)는 그 함수 안에서만 이용 가능
 - 함수 안의 블록 안에서 선언된 변수(지역변수)는 그 블록 안에서만 이용 가능
 - 함수 바깥 부분은 아무런 블록으로도 묶여있지 않기 때문에 함수 밖에서, 즉 global 지역에서 선언된 변수(전역변수)는 프로그램 전체에서 이용 가능

Scope

■ 전역 변수

- 함수 외부에 정의되는 변수
- main(void)도 함수이기 때문에 main(void)함수 밖에 정의
- 다른 말로 외부(external) 변수라고 한다

■ 지역 변수

- 함수 내에서 정의한 변수
- 정의한 함수 내에서만 사용 가능
- 정적 변수 : 함수 호출한 뒤, 다시 함수 호출하는 사이에 변수 값 유지
- 자동 변수 : 지역 변수가 함수를 호출할 때마다 새롭게 만들어 지고, 함수 실행이 끝나면 변수가 없어짐

Scope

■ 예제 프로그램- scope rule

```
1 #include <stdio.h>
2
3 int a=10, b=11, c=12;
4
5 int main(void)
6 {
7     int a=1, b=2;
8     printf("%d %d %d\n", a, b, c);
9     {
10         int b=4;
11         float c=5.0;
12         printf("%d %d %f\n", a, b, c);
13         a=b;
14         {
15             int c;
16             c=b;
17             printf("%d %d %d\n", a, b, c);
18         }
19         printf("%d %d %f\n", a, b, c);
20     }
21     printf("%d %d %d\n", a, b, c);
22     return 0;
23 }
```

```
[root@mclab chap4]# vi chap4-5.c
[root@mclab chap4]# gcc -o chap4-5 chap4-5.c
[root@mclab chap4]# ./chap4-5
1 2 12
1 4 5.000000
4 4 4
4 4 5.000000
4 2 12
[root@mclab chap4]#
```

block 1

a, b : 각각 block1 내부의 a, b를 가리킨다.
c : block1 내부에 변수 c가 없으므로 그 상위 영역인 전역변수 c를 보게 된다.

block 2

a : block2 내부에 변수 a가 없으므로 그 상위영역인 block1의 변수 a를 보게 된다. 그러므로 block2에서의 a의 값의 변경은 block1의 변수 a의 값의 변경을 의미한다.
b, c : 각각 block2 내부의 a, b를 가리킨다.

block 3

a, b : block3 내부에 변수 a, b가 없으므로 상위영역의 변수를 가리키게 된다. (a는 block1의 a를, b는 block2의 b를 가리킴)
c : block 3 내부의 c를 가리킨다. c의 변경은 상위 영역의 c의 변화를 초래하지 않는다.

Scope

■ 예제 프로그램 - scope rule (Parallel and Nested Blocks)

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     int a=1, b=2;
6     {
7         float b=3.0;
8         printf("a=%d, b=%f\n", a, b);
9     }
10
11     {
12         float a=4.0;
13         printf("a=%f, b=%d\n", a, b);
14     }
15 }
```

block 1

□ int a는 보이지만 int b는 볼 수 없다. 대신 float b가 보인다.

block 2

□ int b는 보이지만 int a는 볼 수 없다. 대신 float a가 보인다. Block 1의 float b는 block 2에서는 볼 수 없다. (block 1과 block 2가 parallel한 관계에 있기 때문)

```
[root@mclab chap4]# vi chap4-6.c
[root@mclab chap4]# gcc -o chap4-6 chap4-6.c
[root@mclab chap4]# ./chap4-6
a=1, b=3.000000
a=4.000000, b=2
[root@mclab chap4]#
```

Scope

```
/* This is a sample to demonstrate scope. The techniques
   used in this program should never be used in practice.
*/
```

```
#include <stdio.h>
```

```
int fun (int a, int b);
```

Global area

```
int main (void)
```

```
{
```

```
    int    a;
```

```
    int    b;
```

```
    float  y;
```

```
    ...
```

```
    { // Beginning of nested block
```

```
        float a = y / 2;
```

```
        float y;
```

```
        float z;
```

```
        ...
```

```
        z = a * b;
```

```
        ...
```

```
    } // End of nested block
```

```
    ...
```

```
} // End of main
```

main's area

Nested block
area

```
int fun (int i, int j)
```

```
{
```

```
    int a;
```

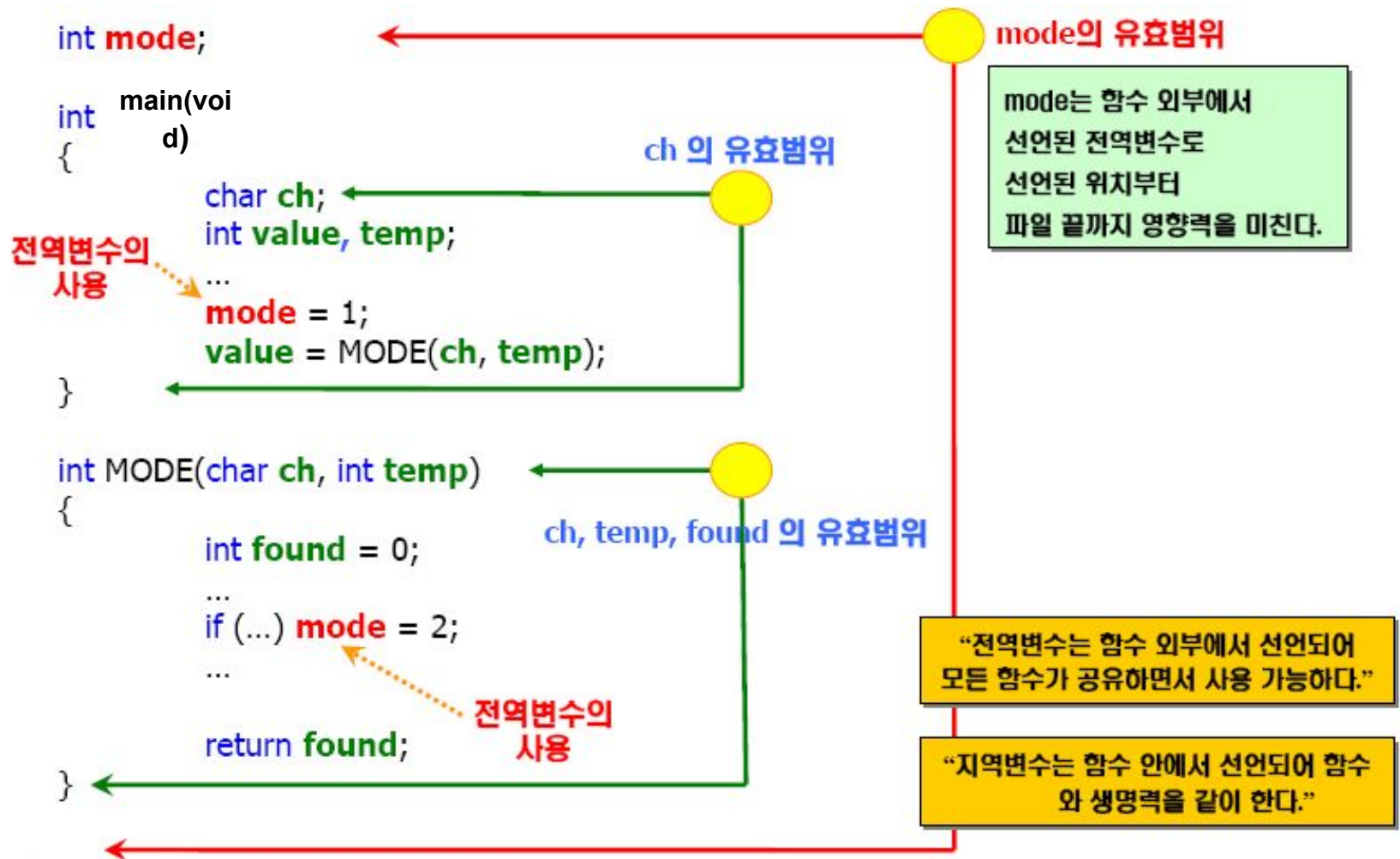
```
    int y;
```

```
    ...
```

```
} // fun
```

fun's area

Scope



예제 프로그램 3 - 나눗셈

예제 소스

```
#include <stdio.h>

void Calc_divide(int in1, int in2);
int Calc_remainder(int in1, int
in2);
int result;

int
main(void)
{
    int intNum1;
    int intNum2;
    int intCalc;

    printf("Enter two integral number: ");
    scanf("%d %d", &intNum1, &intNum2);

    Calc_divide(intNum1, intNum2);
    intCalc = result;
    printf("%d / %d is %d", intNum1, intNum2, intCalc);

    intCalc = Calc_remainder(intNum1, intNum2);
    printf(" with a remainder of: %d\n", intCalc);

    return 0;
}

void Calc_divide(int in1, int in2);
{
    result = in1 / in2;
}

int Calc_remainder(int in1, int in2)
{
    int result_remainder;
    result_remainder = in1 % in2;
    return result_remainder;
}
```

전역변수(global variable), 지역변수(local variable)의 위치를 확인하고 변수들의 유효범위(scope)를 확인한다.

```
kimjihwan@csp:~/test_dr$ gcc -o calc.o calc.c
kimjihwan@csp:~/test_dr$ ./calc.o
Enter two integral number: 13 2
13 / 2 is 6 with a remainder of: 1
kimjihwan@csp:~/test_dr$
```

결과

예제 프로그램 3 - 나눗셈

```
#include <stdio.h>

void Calc_divide(int in1, int in2);
int Calc_remainder(int in1, int in2);

int result;

int
main(void)
{
    int intNum1;
    int intNum2;
    int intCalc;

    printf("Enter two integral number: ");
    scanf("%d %d", &intNum1, &intNum2);

    Calc_divide(intNum1, intNum2);
    intCalc = result;
    printf("%d / %d is %d", intNum1, intNum2, intCalc);

    intCalc = Calc_remainder(intNum1, intNum2);
    printf(" with a remainder of: %d\n", intCalc);

    return 0;
}

void Calc_divide(int in1, int in2);
{
    result = in1 / in2;
}

int Calc_remainder(int in1, int in2)
{
    int result_remainder;
    result_remainder = in1 % in2;
    return result_remainder;
}
```

같은 이름의 변수
값은 같을까?

main's area

Calc_remainder's
area

예제 프로그램 3 - 나눗셈

```
#include <stdio.h>

void Calc_divide(int in1, int in2);
int Calc_remainder(int in1, int in2);

int result;

int
main(void)
{
    int intNum1;
    int intNum2;
    int intCalc;

    printf("Enter two integral number: ");
    scanf("%d %d", &intNum1, &intNum2);

    Calc_divide(intNum1, intNum2);
    intCalc = result;
    printf("%d / %d is %d", intNum1, intNum2, intCalc);

    intCalc = Calc_remainder(intNum1, intNum2);
    printf(" with a remainder of: %d\n", intCalc);

    return 0;
}

void Calc_divide(int in1, int in2);
{
    result = in1 / in2;
}

int Calc_remainder(int in1, int in2)
{
    int result_remainder;
    result_remainder = in1 % in2;
    return result_remainder;
}
```

standard function을
사용하기 위해 또 다른
어떤 **header file**이
있을까?

실습 11

실습 12